Where JopeDime's evolved models win

Prediction programs tuned to what the business actually pays for

Case-study brief • every score cross-validated on training data • 2026-11-03T19:20:45Z

The short version

Most machine-learning contests reward accuracy and nothing else. A business pays for more than accuracy: for the computer time it takes to retrain a model, for how fast it answers, and for how much it costs to run. JopeDime lets an evolutionary search write prediction programs and judges every candidate on a **business scorecard** that includes those costs.

We ran 14 business cases. In each, we compared our evolved programs with well-known alternatives: in this case, 14 of them on all 5 folds (including XGBoost, LightGBM, CatBoost, tuned versions of the most popular libraries, and classic statistical methods) and 3 slower ones (AutoGluon, TabPFN and Kumo) on one fold. We count a case only if we win it *clearly*: no alternative comes close on that case's scorecard. **One case passes that test**, and this brief is about it.

Hourly electricity-price forecasting, with every retrain billed by CPU time

24 %	**27 %**	**8 ×**
better business score than the best alternative (XGBoost), for our typical run	better for our best run, which wins on 5 of 5 data folds	less training CPU per refit than XGBoost (0.38 s vs 0.048 s), for our best program

How we kept it fair

- **The same data and the same scorecard for everyone.** Every model, ours and the alternatives, is fitted on the same rows and scored by the same program.
- **Cross-validation on the training data only.** The training data is split into 5 parts ("folds"). Each model is trained on 4 parts and scored on the fifth, five times over, and the score is the average. The separate test data was never used.
- **Every number comes from the run records.** The brief is regenerated from them by a script; nothing is typed by hand.

The case: forecasting tomorrow's hourly electricity prices

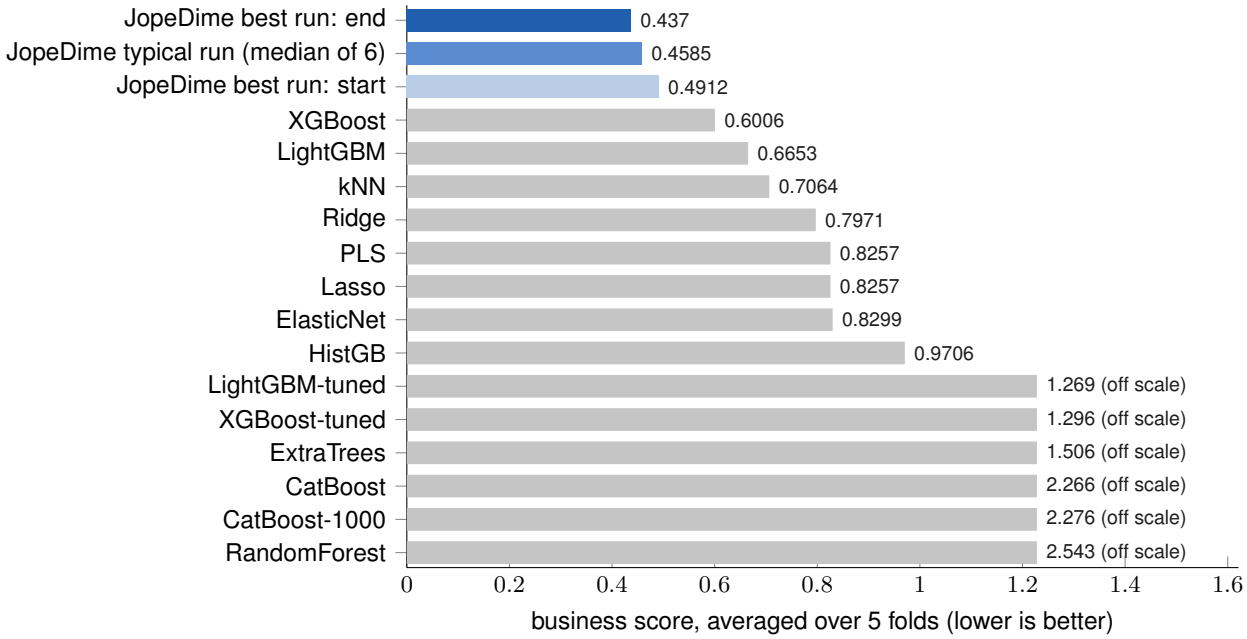
The question

An energy trader re-estimates a price-forecast model every hour, across many markets, and every refit is paid for in computer time. *Which model gives the best forecasts once you also pay for the computer time it takes to retrain?* The data are 6,720 hours of day-ahead electricity prices with 11 inputs (the market, the time, the day of the week and two outside variables), from a public benchmark collection (Nixtla).

The scorecard

Score = forecast error + a charge for training time. The error is measured relative to how much prices normally move. The charge adds a tenth of a typical price swing for every 0.22 seconds of CPU time a retrain uses. A model that forecasts equally well but retrains in a fifth of the time therefore gets the better score. **Lower is better.**

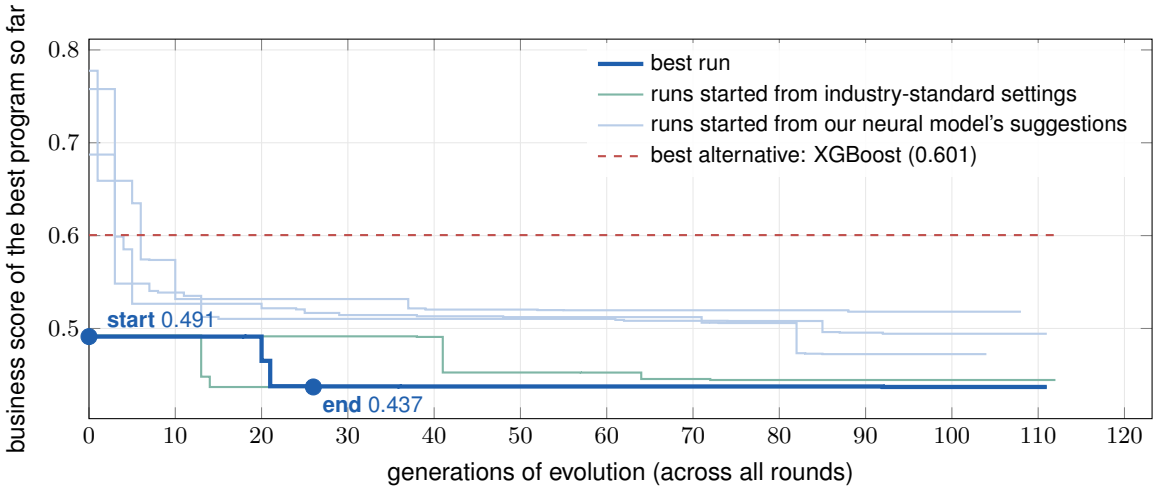
How everyone scored



Grey: 14 alternatives, each scored on all 5 folds ("tuned" versions searched 40 settings inside every training fold). Light blue: where our best run *started*, from our engine's version of XGBoost's default settings. Blue: our typical run (the middle of 6 runs). Dark blue: our best run. Very large scores are cut at the edge of the chart, with their value written beside them.

3 slower alternatives were scored on one fold only, so they are not in the chart. On that fold they scored AutoGluon 379, Kumo-Tabular 0.895, TabPFN-v2 0.392, against 0.287 for our best program on the same fold.

How evolution improved the score



Each line is one run of the search: six runs in all (two ways of choosing the starting programs × three random seeds), each of up to six rounds. A line drops each time the run finds a better program, scored over all 5 folds. Together the runs evaluated about 20,800 programs.

Where the best run started, and where it ended up

Where it started (generation 0)

Boosted decision trees with XGBoost's standard settings, as our engine runs them.

business score	0.491
forecast error (RMSE)	0.0142
training CPU per refit	0.075 s
model size	442 KB

```
(pipe X (histgbm bins=255 depth=6 lambda=1 lr=0.3
min_leaf=1 rounds=100))
```

Where it ended up (found in generation 26)

What changed: the same kind of model (boosted decision trees) with new settings: histogram bins 255 → 256, tree depth 6 → 4.

business score	0.437
forecast error (RMSE)	0.0129
training CPU per refit	0.048 s
model size	139 KB

```
(pipe X (histgbm bins=256 depth=4 lambda=1
lr=0.3 min_leaf=1 rounds=100))
```

What this means

- **More accurate and cheaper at the same time.** Our best program's forecast error (0.0129) is lower than that of every alternative scored on all 5 folds (the best of them, XGBoost: 0.0132). It also retrains with about 8 times less CPU time than XGBoost.
- **Two sources of the win.** Our engine already runs a standard boosted-tree model with far less CPU than the Python libraries do: the starting point beats XGBoost on its own. Evolution then found settings that are both more accurate and cheaper still.
- **Tuning the libraries does not close the gap.** Searching 40 settings for XGBoost or LightGBM inside every fold gives scores near 1.30: the extra training time costs more than the accuracy gained.

Keep in mind

- **The CPU price is part of the case.** It was fixed before any run, not taken from a real tariff. Our best program is also the most accurate of the fully cross-validated models, so it stays ahead whenever computer time costs anything at all. Our typical run's error (0.0132) is about the same as XGBoost's (0.0132); its lead comes from the CPU saving.
- **The most accurate model of all was much costlier.** On the one fold where AutoGluon was run, its 5-minute ensemble had a somewhat lower error (0.00746 vs our 0.00819), but it used about 830 CPU-seconds per refit.
- **The folds are shuffled hours.** Like every model here, ours was checked on randomly chosen hours. That measures how well a model fits the period, not how it forecasts a future it has not seen.
- **The other 13 cases.** We do not claim them. Under the same cross-validation either an alternative won, or our lead was within the normal variation between folds.

Behind the numbers. Our runs: `extend_case.sh --cuts 5`, run stamp `20261103T170155Z-cv5`, case 02. Alternatives: `summary_cv.csv`, scored by `jopedime fitness evaluate`. Built by `make -C docs/case-study-brief` from 225 evidence files (digest `e5504c3944eafd88`). The full technical record of all 14 cases is `campaign-technical.pdf`.